A Progress (Review-1) REPORT ON

Parameterized Matrix Multiplier Accelerator with UVM Verification

SUBMITTED BY

PRN	Student Name
23070123109	Saaj Mulik
23070123120	Shloka Patel
23070123123	Shrey Raj
23070123165	Swayam Patel

Under the Guidance of

Dr. Suman Lata Tripathi

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

[(A CONSTITUTENT OF SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)]

Pune – 412115

Date of Submission: 22/08/2026

I: Introduction

Modern artificial intelligence applications — from image recognition to natural language processing — depend fundamentally on matrix multiplication. Every layer of a neural network computes a dot product between weight matrices and input vectors, making matrix multiplication the single most frequently executed operation in AI workloads. A general-purpose CPU executes this computation sequentially, processing one multiply-accumulate operation at a time. For large matrices encountered in production AI systems, this sequential approach results in high latency and excessive power consumption.

This project proposes to address this bottleneck by designing a Parameterized Matrix Multiplier Accelerator — a piece of custom hardware implemented in SystemVerilog that performs N×N matrix multiplication entirely in parallel. The design is based on the systolic array architecture, the same computational fabric that powers Google's Tensor Processing Unit (TPU). In a systolic array, an N×N grid of Processing Elements (PEs) — each performing a single Multiply-Accumulate (MAC) operation per clock cycle — work simultaneously, reducing the total latency from $O(N^3)$ sequential operations to just 2N–1 clock cycles.

I.I Objectives:

- Design a parameterized N×N matrix multiplier in SystemVerilog using MAC units tiled into a systolic array
- Verify functional correctness using a complete UVM testbench with a Python NumPy golden model
- Achieve greater than 90% functional coverage before hardware deployment
- Synthesize the design in Xilinx Vivado, collect LUT/DSP/Fmax metrics, and deploy on FPGA
- Demonstrate a neural network inference layer (Output = Weight × Input) on real FPGA hardware

I.II **Tools and Technologies:**

Category	Tool/Technology
HDL	SystemVerilog
Simulation	EDA Playground/ Cadence NCSim
Waveform+Schematic	EDA Playground/ Cadence NCSim
Verification	UVM, SystemVerilog Assertions, Functional Coverage
Golden Model	Python 3 +NumPy
FPGA Synthesis	Xilinx Vivado

I.III System Architecture:

The accelerator is built bottom-up from three levels: the MAC unit (Processing Element), the systolic array (top-level RTL), and the testbench environment. This section describes the architecture at each level.

- MAC Unit - Processing Element

  The MAC unit is the fundamental computational building block. Each clock cycle, it performs one multiply-accumulate operation: it takes one element from matrix A and one from matrix B, multiplies them, adds the product to a running sum, and passes A rightward and B downward to neighbouring PEs. The module is parameterized by data width W (default 8 bits), making it reusable across different precision requirements.

  Operation per clock cycle: sum_out = sum_in + (a_in × b_in)

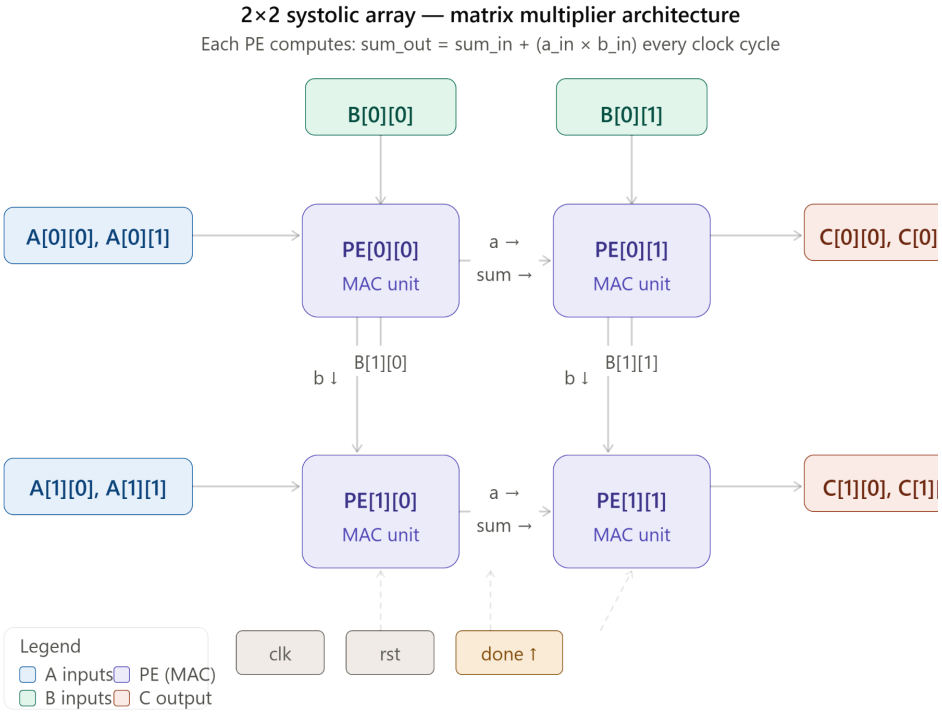
- 2x2 Systolic Array

  For the current implementation, four MAC units are arranged in a 2×2 grid. Inputs are applied as rows of A and columns of B fed into the array. The systolic rhythm ensures that each PE accumulates the correct partial product over N clock cycles. For a 2×2 multiplier, the final output is available after 3 clock cycles (2N–1 = 3 for N=2). The design includes a clocked always block, synchronous reset (rst), and a "done" signal that asserts when the output is valid.

  Key signals: clk, rst, A[0:1][0:1] (8-bit), B[0:1][0:1] (8-bit), C[0:1][0:1] (16-bit),

  done.

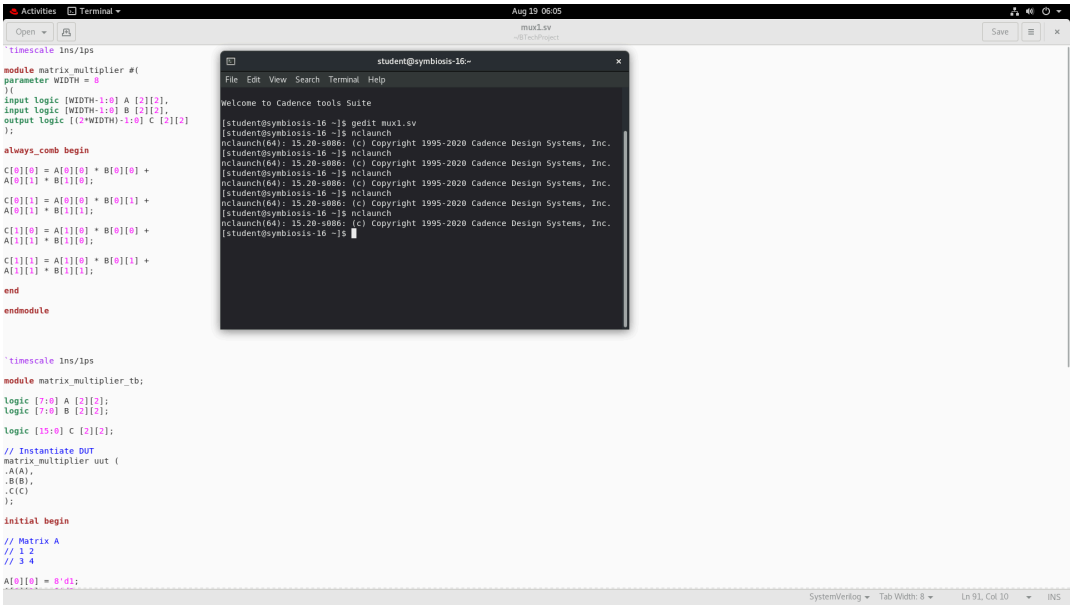
- Architecture Diagram

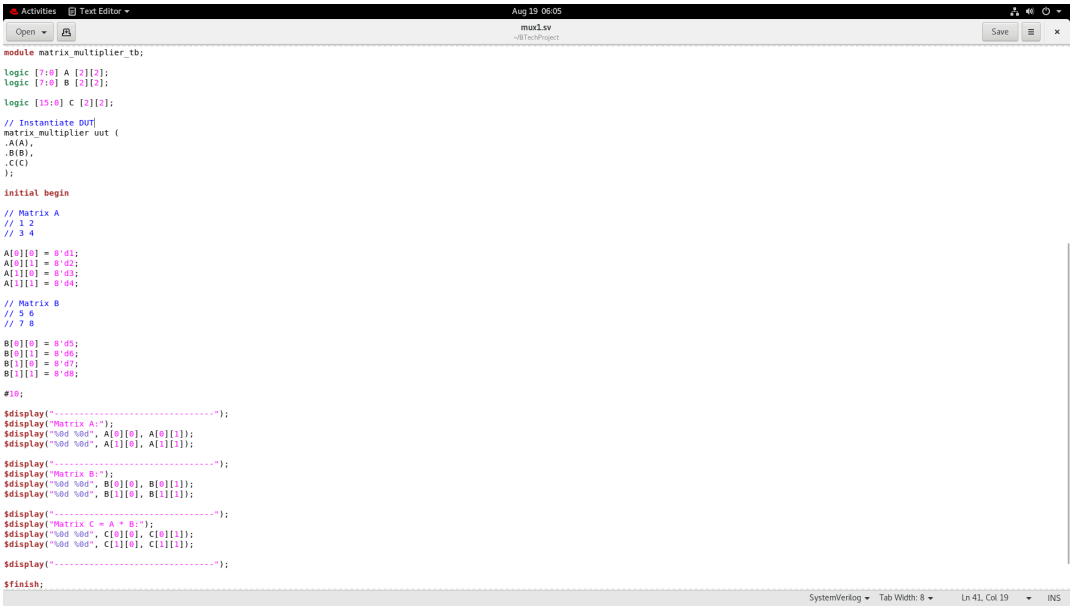


II: Project Progress details

Week	Work Done
1	Study of project title
2-3	RTL Foundations
4	MAC Unit design and Directed Testbench
5	2x2 Matrix Multiplier with 8 directed tests
6	Cadence NCSim and SimVision Simulation

- Code, output waveform and simulation of matric multiplier on NCSim


Activities
Terminal
Aug 19 06:05
Open
mux1.sv
~/BTechProject
Save
`timescale 1ns/1ps
module matrix_multiplier #(
parameter WIDTH = 8
)(
input logic [WIDTH-1:0] A [2][2],
input logic [WIDTH-1:0] B [2][2],
output logic [(2*WIDTH)-1:0] C [2][2]
);
always_comb begin
C[0][0] = A[0][0] * B[0][0] +
A[0][1] * B[1][0];
C[0][1] = A[0][0] * B[0][1] +
A[0][1] * B[1][1];
C[1][0] = A[1][0] * B[0][0] +
A[1][1] * B[1][0];
C[1][1] = A[1][0] * B[0][1] +
A[1][1] * B[1][1];
end
endmodule
student@symbiosis-16:~
File Edit View Search Terminal Help
Welcome to Cadence tools Suite
[student@symbiosis-16 ~]$ gedit mux1.sv
[student@symbiosis-16 ~]$ nclaunch
nclaunch(64): 15.20-s086: (c) Copyright 1995-2020 Cadence Design Systems, Inc.
[student@symbiosis-16 ~]$ nclaunch
nclaunch(64): 15.20-s086: (c) Copyright 1995-2020 Cadence Design Systems, Inc.
[student@symbiosis-16 ~]$ nclaunch
nclaunch(64): 15.20-s086: (c) Copyright 1995-2020 Cadence Design Systems, Inc.
[student@symbiosis-16 ~]$ nclaunch
nclaunch(64): 15.20-s086: (c) Copyright 1995-2020 Cadence Design Systems, Inc.
[student@symbiosis-16 ~]$
`timescale 1ns/1ps
module matrix_multiplier_tb;
logic [7:0] A [2][2];
logic [7:0] B [2][2];
logic [15:0] C [2][2];
// Instantiate DUT
matrix_multiplier uut (
.A(A),
.B(B),
.C(C)
);
initial begin
// Matrix A
// 1 2
// 3 4
A[0][0] = 8'd1;
SystemVerilog
Tab Width: 8
Ln 91, Col 10
INS


Activities
Text Editor
Aug 19 06:05
Open
mux1.sv
~/BTechProject
Save
module matrix_multiplier_tb;
logic [7:0] A [2][2];
logic [7:0] B [2][2];
logic [15:0] C [2][2];
// Instantiate DUT
matrix_multiplier uut (
.A(A),
.B(B),
.C(C)
);
initial begin
// Matrix A
// 1 2
// 3 4
A[0][0] = 8'd1;
A[0][1] = 8'd2;
A[1][0] = 8'd3;
A[1][1] = 8'd4;
// Matrix B
// 5 6
// 7 8
B[0][0] = 8'd5;
B[0][1] = 8'd6;
B[1][0] = 8'd7;
B[1][1] = 8'd8;
#10;
$display("--------------------------------");
$display("Matrix A:");
$display("%0d %0d", A[0][0], A[0][1]);
$display("%0d %0d", A[1][0], A[1][1]);
$display("--------------------------------");
$display("Matrix B:");
$display("%0d %0d", B[0][0], B[0][1]);
$display("%0d %0d", B[1][0], B[1][1]);
$display("--------------------------------");
$display("Matrix C = A * B:");
$display("%0d %0d", C[0][0], C[0][1]);
$display("%0d %0d", C[1][0], C[1][1]);
$display("--------------------------------");
$finish;
SystemVerilog
Tab Width: 8
Ln 41, Col 19
INS

```
logic [15:0] C [2][2];

// Instantiate DUT
matrix_multiplier uut (
.A(A),
.B(B),
.C(C)
);

initial begin

// Matrix A
// 1 2
// 3 4

A[0][0] = 8'd1;
A[0][1] = 8'd2;
A[1][0] = 8'd3;
A[1][1] = 8'd4;

// Matrix B
// 5 6
// 7 8

B[0][0] = 8'd5;
B[0][1] = 8'd6;
B[1][0] = 8'd7;
B[1][1] = 8'd8;

#10;

$display("--------------------------------");
$display("Matrix A:");
$display("%0d %0d", A[0][0], A[0][1]);
$display("%0d %0d", A[1][0], A[1][1]);

$display("--------------------------------");
$display("Matrix B:");
$display("%0d %0d", B[0][0], B[0][1]);
$display("%0d %0d", B[1][0], B[1][1]);

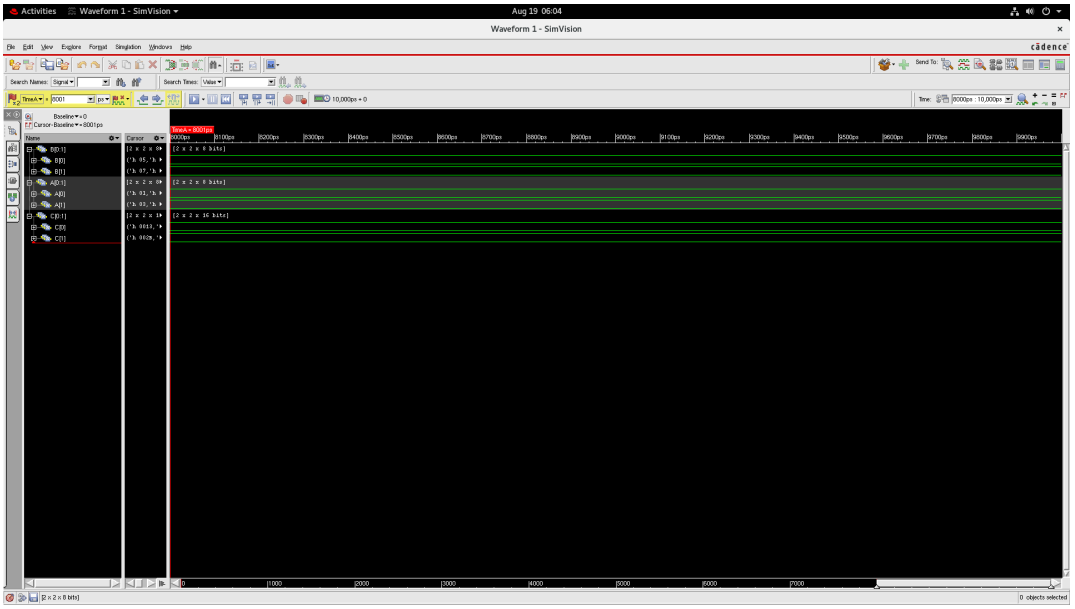
$display("--------------------------------");
$display("Matrix C = A * B:");
$display("%0d %0d", C[0][0], C[0][1]);
$display("%0d %0d", C[1][0], C[1][1]);

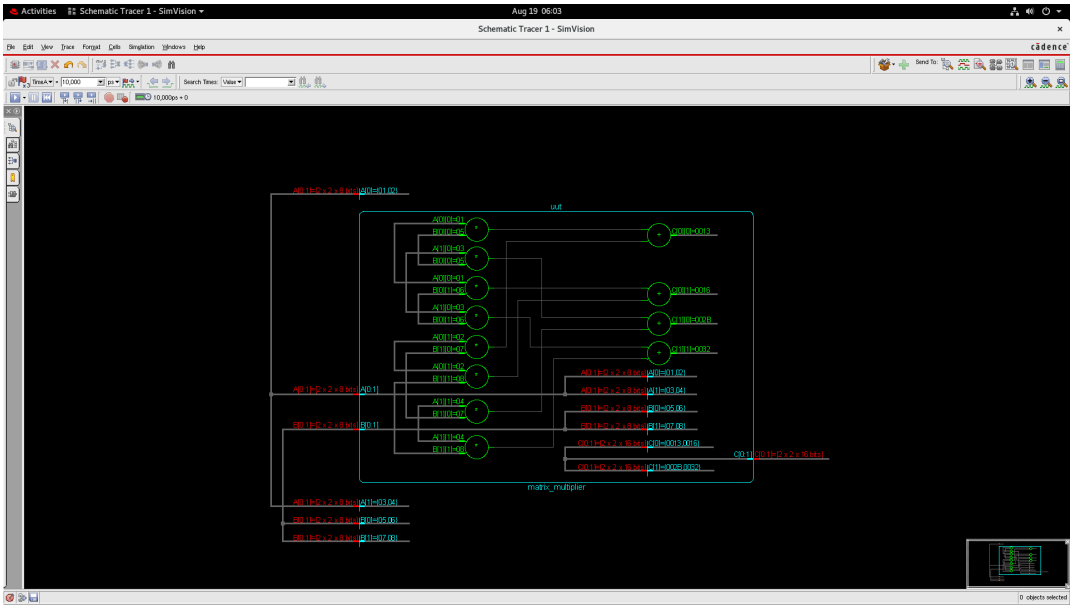
$display("--------------------------------");

$finish;

end

endmodule
```





III: References

[i] ChipVerify (UVM Verification) - https://chipverify.com/tutorials/uvm

[ii] Cadence NCSim user guide - https://community.cadence.com/cadence_technology_forums/f/functional-verification/35689/tutorial-on-ncsim